

I2S Audio Playback Experiment

I2S Audio Playback Experiment

1. Audio Playback Overview
 2. Core Concepts
 - 2.1 I2S Audio Bus
 - 2.2 ES8311 Codec (DAC Mode)
 - 2.3 WAV File Format
 - 2.4 SPIFFS Flash Filesystem
 - 2.5 WS2812 RGB LED
 3. Hardware Features and Dependencies
 - 3.1 ESP32-S3 I2S Audio Interface Features
 - 3.2 ES8311 Audio Codec Specifications
 - 3.3 Pin Assignment Table
 - 3.4 Arduino IDE Configuration
 - 3.5 Library Dependencies
 4. Program Flow
 - 4.1 Overall Flowchart
 - 4.2 Playback Data Flow
 - 4.3 File Structure
 5. Source Code Details
 - 5.1 app_speech.h — Macro Configuration and Interface
 - 5.2 app_speech.cpp — Audio Subsystem Initialization
 - I2C Bus Initialization and Device Scan
 - I2S Driver Initialization
 - ES8311 I2C Read/Write Wrappers
 - ES8311 Hardware Reset
 - DAC Initialization (Anti-Pop Complete Sequence)
 - Main Initialization Entry
 - 5.3 wav_file_player.cpp — WAV Player
 - 5.3.1 Initialization and Corrupted File Detection
 - 5.3.2 Playback Task (wav_play_task)
 - 5.3.3 Serial Upload Listener
 - 5.3.4 Public Interface
 - 5.4 rgb_led.cpp — WS2812 RGB LED
 - 5.5 Main Program — Button State Machine and LED Effects
 6. Operation Flow
 - 6.1 IDE Configuration and Build
 - 6.2 Normal Playback
 - 6.3 Replace Playback Files
 - 6.4 Edge Cases
 7. FAQ
 - Appendix A: ES8311 Register Quick Reference (DAC Playback)
 - Appendix B: Code Index
-

1. Audio Playback Overview

This experiment implements a **ESP32-S3-based I2S audio player**, driving an ES8311 audio codec (DAC mode) via the I2S bus, reading WAV files from SPIFFS flash filesystem, and driving a speaker through an NS4150 power amplifier. Playback control is via the BOOT button (short press: play/pause, long press: stop), and WS2812 RGB LED provides rainbow light effect feedback.



Function	Description
File Storage	SPIFFS flash filesystem, stores WAV audio files
Audio Decoding	ES8311 DAC, 16-bit / 16kHz / stereo output
Playback Control	BOOT button (GPIO0): short press play/pause, long press stop
LED Feedback	WS2812 RGB LED: rainbow hue animation during playback
Amplifier	NS4150 Class D amp, GPIO7 enable
Serial Upload	Supports uploading WAV files to SPIFFS via serial (no recompilation needed)

2. Core Concepts

2.1 I2S Audio Bus

I2S (Inter-IC Sound) is a standard digital audio transmission protocol, containing the following signal lines:

Signal	Full Name	Function
MCLK	Master Clock	Master clock, = sample rate × 256
BCLK	Bit Clock	Bit clock, one pulse per bit of data
WS / LRCLK	Word Select	Frame clock, low = left channel, high = right channel
DOUT	Data Out	Transmit data (ESP32 → DAC)
DIN	Data In	Receive data (ADC → ESP32), this project only uses DOUT

I2S uses Philips standard format (left-justified, MSB first), WS toggles one BCLK before the data MSB.

In Arduino, I2S is operated via `ESP_I2S.h`, initialization requires only 2 lines:

```
I2S.setPins(I2S_BCLK, I2S_WS, I2S_DOUT, -1, I2S_MCLK);
I2S.begin(I2S_MODE_STD, SAMPLE_RATE, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO);
```

2.2 ES8311 Codec (DAC Mode)

ES8311 is a low-power audio codec from Shunli Semiconductor, configured via I2C register access (address 0x18), with I2S for audio data transmission. This project uses DAC mode, using the `wire` object to manually write ES8311 register operations without additional library dependencies. See section 3.2 for complete specifications.

2.3 WAV File Format

WAV is based on the RIFF specification, with PCM encoding consisting of **44-byte header + raw PCM data**:

offset	Size	Field	Description
0x00	4	"RIFF"	Magic number
0x04	4	file_len	File size - 8
0x08	4	"WAVE"	Format identifier
0x0C	4	"fmt "	Format chunk identifier
0x10	4	16	PCM format chunk size
0x14	2	1	Audio format (1=PCM)
0x16	2	channels	Number of channels (1=mono, 2=stereo)
0x18	4	sample_rate	Sample rate (e.g. 16000 Hz)
0x1C	4	byte_rate	Byte rate
0x20	2	block_align	Frame alignment
0x22	2	bit_depth	Bit depth (16)
0x24	4	"data"	Data chunk identifier
0x28	4	data_len	PCM data byte count

This project's WAV files are 16kHz / 16-bit / mono, software copies to stereo during playback.

2.4 SPIFFS Flash Filesystem

SPIFFS is a SPI Flash filesystem. In Arduino, mount with `SPIFFS.begin(true)` (one line, auto-formats on failure), file operations use the `File` object: `.open()` / `.read()` / `.close()`.

2.5 WS2812 RGB LED

WS2812 is a single-wire cascaded RGB LED, GRB byte order, 24-bit color. This project uses the **Adafruit_NeoPixel** library: `led.setPixelColor()` + `led.show()`, library handles RMT timing internally.

3. Hardware Features and Dependencies

3.1 ESP32-S3 I2S Audio Interface Features

Feature	Parameter
I2S Peripherals	2 (I2S0 / I2S1)
I2C Peripherals	2 (I2C0 / I2C1)
SPI Flash	Up to 16MB

Feature	Parameter
GPIO Matrix	Any GPIO mapped to any peripheral
Arduino Core Version	ESP32 v3.3.8 (based on IDF v5.x)

ESP32-S3 GPIO matrix allows any GPIO to be configured as I2S/I2C function pins, no fixed mapping needed. This project uses I2S0 TX channel for audio output, I2C0 to configure ES8311 registers.

3.2 ES8311 Audio Codec Specifications

Feature	Parameter
I2C Address	0x18 (7-bit, AD0=0, AD1=0)
DAC SNR	95dBA
DAC Sample Rate	8kHz ~ 96kHz
Supported Formats	I2S, Left-Justified, Right-Justified, DSP/PCM
Output Volume	0~100% (Arduino version caps at 75%, corresponding to register value 0~255)
Operating Voltage	1.8V ~ 3.3V (analog), 1.8V (digital)
Operating Mode	DAC / ADC / BOTH (this project uses DAC mode)
Package	QFN-32 (4x4mm)

ES8311 operates in slave mode — BCLK and LRCLK are provided by ESP32-S3 I2S master, MCLK output from ESP32-S3 at 4.096MHz.

Clock Tree: $MCLK = 16000 \times 256 = 4.096\text{MHz}$, $BCLK = MCLK / 4 = 1.024\text{MHz}$, $WS = 16\text{kHz}$. I2S Philips standard format, 16-bit width, stereo slots (mono WAV playback copies to left/right slots in software).

3.3 Pin Assignment Table

Pin	Function	Connection Target	Direction
GPIO42	I2C SCL	ES8311 SCL	Output
GPIO41	I2C SDA	ES8311 SDA	Bidirectional
GPIO15	I2S MCLK	ES8311 MCLK	Output
GPIO16	I2S BCLK	ES8311 BCLK	Output
GPIO18	I2S WS	ES8311 LRCK	Output
GPIO8	I2S DOUT	ES8311 SDIN (DAC)	Output
GPIO7	PA_EN	NS4150 amp enable	Output
GPIO48	WS2812 Data	RGB LED data input	Output

Pin	Function	Connection Target	Direction
GPIO0	BOOT button	Active-low button	Input with pullup

3.4 Arduino IDE Configuration

Configuration	Value	Description
Board	ESP32S3 Dev Module	ESP32-S3 general dev board
USB CDC On Boot	Enabled	Enable USB serial
Flash Size	16MB (128Mb)	Match onboard 16MB SPI Flash
Partition Scheme	Custom (partitions.csv)	Custom partition with SPIFFS
PSRAM	OPI PSRAM	Enable 8MB PSRAM

Custom partition allocates ~3.44MB to SPIFFS: `spiffs, data, spiffs, 0x90000, 0x370000`.

3.5 Library Dependencies

Library	Purpose	Reference Tutorial
ESP32 (by Espressif)	Arduino core (includes ESP_I2S, SPIFFS, Wire, FreeRTOS)	Arduino → 1. Before Development → 1.Environment Setup
Adafruit_NeoPixel	WS2812 RGB LED driver	2.Basic Peripherals → 9.RGB Colorful Light Experiment

ES8311 is fully controlled via I2C register writes, no additional library needed.

4. Program Flow

4.1 Overall Flowchart

```

setup():
  1. digitalWrite(VOICE_EN_PIN, 0)      ← Pull PA low ASAP to suppress power-on pop
  2. Serial.begin(115200)
  3. Codec_I2S_init()                   ← Init audio playback chain
    ├── power_on_codec()                ← PA pin config (keep low)
    ├── i2c_init()                      ← Wire.setPins + wire.begin + I2C scan
    ├── i2s_driver_init()               ← I2S.setPins + I2S.begin
    ├── es8311_hard_reset()             ← ES8311 hardware reset
    └── es8311_init_dac()               ← Mute→Clock→DAC→Vol→Flush→PA→Unmute
  4. wav_player_init()                 ← SPIFFS mount + file scan + corruption check
  5. btn_play_init()                   ← pinMode(BTN_PLAY_PIN, INPUT_PULLUP)
  6. rgb_led_init(); rgb_led_off()

loop():
```

```
└─ Button detection (1kHz): debounce → timing → short/long judgment
└─ Rainbow LED (~10ms step while playing)
└─ wav_player_check_upload()      ← Background serial upload listener
└─ vTaskDelay(1)
```

4.2 Playback Data Flow

```
SPIFFS file → file.read(mono_buf) → Mono→Stereo copy
           → I2S.write(stereo_buf) → I2S DOUT → ES8311 DAC → NS4150 amp → Speaker
```

4.3 File Structure

```
Arduino_Player_V2/
└─ Arduino_Player_V2.ino      ← Main program: setup() + loop() + button FSM
└─ app_speech.h / .cpp       ← ES8311 driver + I2S init
└─ wav_file_player.h / .cpp  ← SPIFFS mount + playback task + upload
└─ rgb_led.h / .cpp          ← WS2812 driver (Adafruit_NeoPixel)
└─ partitions.csv            ← Partition table (512KB app + 3.44MB SPIFFS)
└─ data/*.wav                ← WAV files to flash
└─ flash_spiffs.bat / .ps1   ← SPIFFS flash scripts
```

5. Source Code Details

Source code: [Source code](#) → [Arduino](#) → [4.Audio](#) → [Arduino_Player_V2](#) → [Arduino_Player_V2.ino](#)

5.1 app_speech.h — Macro Configuration and Interface

`app_speech.h` is the header file for the entire audio playback subsystem, uniformly defining I2C/I2S pin mappings, audio sampling parameters, and external interface function declarations. All macros are centrally managed for easy hardware pin changes.

```
/* ===== Pins ===== */
#define I2C_SCL_PIN      42      // I2C clock
#define I2C_SDA_PIN      41      // I2C data
#define I2S_MCLK          15      // I2S master clock
#define I2S_BCLK          16      // I2S bit clock
#define I2S_WS            18      // I2S word select
#define I2S_DOUT          8       // I2S data out (TX)
#define I2S_DIN           -1      // Unused
#define VOICE_EN_PIN      7       // PA enable

/* ===== Audio ===== */
#define SAMPLE_RATE       16000   // 16kHz sample rate
#define MCLK_MULTIPLE     256     // MCLK = 16000 x 256 = 4.096MHz
#define VOICE_VOLUME      75      // Volume 0-100 (ES8311 max 75)
#define BUF_SIZE          1024    // Playback buffer size (bytes)

/* ===== Button ===== */
#define BTN_PLAY_PIN      0       // BOOT button (active-low)
```

```

/* ===== API | Public API ===== */
void Codec_I2S_init(void); // Full audio system init
void app_speech_play(uint8_t *buf, uint32_t len); // Write PCM to I2S
void es8311SetMute(bool mute); // Mute control
void paEnable(bool enable); // PA enable

```

5.2 app_speech.cpp — Audio Subsystem Initialization

`Codec_I2S_init()` initializes the entire playback chain in sequence, with the power amplifier enabled last to avoid power-on pop noise.

I2C Bus Initialization and Device Scan

I2C uses Arduino standard `wire` library, after `wire.setPins()` specifies pins, `wire.begin()` starts the bus. After initialization, scan addresses 1~126 to confirm ES8311 responds at 0x18.

```

#include <Wire.h> // Arduino standard I2C library
#include <ESP_I2S.h> // ESP32 Arduino I2S wrapper

I2SClass I2S; // Global I2S object

static void i2c_init(void) {
    wire.setPins(I2C_SDA_PIN, I2C_SCL_PIN); // Assign I2C pins
    wire.begin(); // Start I2C (default 100kHz)

    // Scan 1~126, confirm ES8311 at 0x18
    for (uint8_t addr = 1; addr < 127; addr++) {
        wire.beginTransaction(addr);
        if (wire.endTransmission() == 0) // Returns 0 = ACK = device present
            Serial.printf(" [OK] 0x%02x\n", addr);
    }
}

```

I2S Driver Initialization

`I2S.setPins()` configures BCLK/WS/DOUT/MCLK pins (DIN filled with -1 means unused), `I2S.begin()` starts standard Philips mode, 16kHz sample rate, 16-bit width, stereo slot I2S TX channel. Two lines of code encapsulate underlying channel creation, clock calculation, and GPIO binding.

```

static void i2s_driver_init(void) {
    // BCLK, WS, DOUT, DIN(-1=unused), MCLK
    I2S.setPins(I2S_BCLK, I2S_WS, I2S_DOUT, -1, I2S_MCLK);
    if (!I2S.begin(I2S_MODE_STD, SAMPLE_RATE, I2S_DATA_BIT_WIDTH_16BIT,
        I2S_SLOT_MODE_STEREO)) {
        Serial.println("[I2S] ERROR: Init failed");
        return;
    }
    delay(100);
}

```

ES8311 I2C Read/Write Wrappers

All ES8311 register operations are built on these two functions. `es_write()` sends register address and value then generates STOP to end transmission; `es_read()` first writes register address (no STOP), then initiates read request to get 1 byte of data.

```
/* Write one byte to register */
static void es_write(uint8_t reg, uint8_t val) {
    wire.beginTransaction(ES8311_ADDR);
    wire.write(reg); wire.write(val);          // Send register address + value
    wire.endTransmission();                    // Generate STOP
}

/* Read one byte from register */
static uint8_t es_read(uint8_t reg) {
    wire.beginTransaction(ES8311_ADDR);
    wire.write(reg);
    wire.endTransmission(false);              // No STOP, keep bus
    wire.requestFrom((uint8_t)ES8311_ADDR, (uint8_t)1);
    return wire.available() ? wire.read() : 0;
}
```

ES8311 Hardware Reset

After ES8311 powers on, a complete hardware reset sequence must be executed, including analog power enable, clock tree initialization, slave mode setting, external MCLK selection, and ADC/DAC filter configuration. Register write order strictly follows chip manual requirements.

```
static void es8311_hard_reset(void) {
    es_write(REG0D, 0xFA); es_write(REG44, 0x08); es_write(REG44, 0x08);
    es_write(REG01, 0x30); es_write(REG02, 0x00); es_write(REG03, 0x10);
    es_write(REG04, 0x10); es_write(REG05, 0x00); es_write(REG16, 0x24);
    es_write(REG0B, 0x00); es_write(REG0C, 0x00); es_write(REG10, 0x1F);
    es_write(REG11, 0x7F); es_write(REG00, 0x80);

    es_write(REG00, es_read(REG00) & 0xBF);          // Slave mode
    es_write(REG01, 0x3F & 0x7F);                    // External MCLK
    es_write(REG06, es_read(REG06) & ~0x20);
    es_write(REG13, 0x10); es_write(REG1B, 0x0A); es_write(REG1C, 0x6A);
    es_write(REG44, 0x58);

    delay(10);
}
```

DAC Initialization (Anti-Pop Complete Sequence)

This is the most critical function for eliminating power-on/startup pop noise. The core idea is to ensure that when the power amplifier turns on, the speaker receives a mute signal instead of random noise. The 7-step sequence is interlocked with strict order:

```
static void es8311_init_dac(void) {
```



```

    es8311SetMute(true);           // 1. Mute first
    es8311_set_format_clock();      // 2. I2S clock config
    es8311_start_dac();            // 3. Start DAC (still muted)
    es8311SetVolume(VOICE_VOLUME); // 4. Set volume
    // 5. Flush I2S with silence
    uint8_t silence[320] = {0};
    I2S.write(silence, sizeof(silence)); // Write 320 bytes of 0 (40ms
silence)
    // 6. PA on (no noise, still muted)
    paEnable(true);
    delay(50);
    // 7. Unmute – pop already suppressed
    es8311SetMute(false);
}

```

Mute → configure clock → start DAC → flush silence → enable PA → unmute. When the power amplifier turns on, the speaker receives a silent signal, so the pop is naturally eliminated.

Main Initialization Entry

`Codec_I2S_init()` chains all above steps into a complete audio chain initialization sequence. Order is critical: I2S must start before ES8311 (ES8311 is slave, depends on ESP32 for MCLK/BCLK/WS); PA is enabled last, ensuring the entire chain outputs stable silence when the amplifier turns on.

```

void Codec_I2S_init(void) {
    Serial.println("==== Audio System Init (Playback Only) =====");

    power_on_codec();           // 1. PA power preparation
    i2c_init();                 // 2. I2C bus + scan
    i2s_driver_init();          // 3. I2S start (MCLK must precede ES8311)
    es8311_hard_reset();        // 4. ES8311 hardware reset
    delay(50);
    es8311_init_dac();          // 5. DAC init

    Serial.println("==== Audio Player Ready =====");
}

```

5.3 wav_file_player.cpp — WAV Player

Responsible for SPIFFS mounting, file scanning and verification, playback task management, and serial upload monitoring.

5.3.1 Initialization and Corrupted File Detection

`wav_player_init()` mounts SPIFFS (auto-formats on failure), then scans all files and lists them. For .wav files, additionally reads the `data_len` field from the header and compares it against the actual file size — incomplete files are automatically marked and deleted to prevent corrupted files from serial upload interruptions.

```

#include <SPIFFS.h>
#include <freertos/FreeRTOS.h>

```

```

#include <freertos/task.h>

static volatile bool s_is_playing = false;
static volatile bool s_active     = false;
static volatile bool s_paused     = false;
static volatile bool s_stop       = false;

static uint8_t mono_buf[BUF_SIZE];           // Mono read buffer
static uint8_t stereo_buf[BUF_SIZE * 2];     // Stereo expansion buffer

void wav_player_init(void) {
    if (!SPIFFS.begin(true)) {                // auto-format on mount failure
        Serial.println("[SPIFFS] ERROR: Mount failed");
        return;
    }
    Serial.printf("[SPIFFS] %uKB total / %uKB used\n",
        SPIFFS.totalBytes() / 1024, SPIFFS.usedBytes() / 1024);

    // Scan all files + check WAV integrity
    File root = SPIFFS.open("/");
    File f     = root.openNextFile();
    while (f) {
        // For .wav files: compare header data_len against actual file size
        // Incomplete files auto-detected and deleted
        ...
    }
}

```

Initialization auto-detects WAV file integrity — compares header `data_len` with actual file size, auto-deletes incomplete files. Especially useful for files that may be truncated after serial uploads.

5.3.2 Playback Task (wav_play_task)

Playback task runs as a FreeRTOS Task (8KB stack, priority 5), core flow in three phases: (1) Open file and parse WAV header, verify RIFF magic, PCM format, 16-bit depth; (2) Loop read mono PCM data, copy to stereo, push via `I2S.write()` to DMA; (3) On playback end or stop signal, send mute frames to flush I2S pipeline, close file and self-delete task.

```

#pragma pack(push, 1)                                // 1-byte alignment
typedef struct {
    char    riff[4];      uint32_t file_len;           // "RIFF" + file length
    char    wave[4];      char    fmt[4];             // "WAVE" + "fmt "
    uint32_t fmt_len;      uint16_t audio_fmt;         // 16 + 1(PCM)
    uint16_t channels;     uint32_t sample_rate;       // channels + sample rate
    uint32_t byte_rate;    uint16_t block_align;       // byte rate + block align
    uint16_t bits_per_sample;
    char    data[4];      uint32_t data_len;          // "data" + data length
} wav_header_t;
#pragma pack(pop)

static void wav_play_task(void *arg) {
    const char *path = (const char *)arg;

```

```

do {
    // --- File open & WAV parsing ---
    File f = SPIFFS.open(path, "r");
    if (!f || f.size() == 0) break;

    wav_header_t hdr;
    f.read((uint8_t *)&hdr, sizeof(hdr));           // Read 44-byte WAV header

    // Validate: "RIFF" + "WAVE" + PCM + 16-bit
    if (memcmp(hdr.riff, "RIFF", 4) ||
        memcmp(hdr.wave, "WAVE", 4)) break;
    if (hdr.audio_fmt != 1 || hdr.bits_per_sample != 16) break;

    bool expand = (hdr.channels == 1);               // Mono needs stereo expansion
    s_active = true;

    // --- Core Playback Loop ---
    size_t rd;
    uint32_t total = 0;
    while (!s_stop && (rd = f.read(mono_buf, BUF_SIZE)) > 0) {
        while (s_paused && !s_stop)                 // Pause wait loop
            vTaskDelay(pdMS_TO_TICKS(50));
        if (s_stop) break;

        total += rd;
        if (expand) {
            // Mono → Stereo: {s0,s0, s1,s1, ...}
            int16_t *s = (int16_t *)mono_buf;
            int16_t *d = (int16_t *)stereo_buf;
            size_t cnt = rd / 2;                     // Sample count (16-bit)
            for (size_t i = 0; i < cnt; i++)
                d[i * 2] = d[i * 2 + 1] = s[i];      // Each sample to both slots
            app_speech_play(stereo_buf, cnt * 4);
        } else {
            app_speech_play(mono_buf, rd);            // Already stereo, write directly
        }
        vTaskDelay(pdMS_TO_TICKS(3));               // 3ms CPU yield
    }
    f.close();
} while (false);

// Cleanup: silence → reset → delete task
uint8_t silence[320] = {0};
app_speech_play(silence, sizeof(silence));          // Flush I2S pipeline
s_is_playing = false; s_active = false;
vTaskDelete(NULL);                                 // Task self-delete
}

```

Playback loop key points:

- `do { ... } while(false)` + `break` for early error exit, avoids nested `if/else`
- Mono `{s0}` → stereo `{s0,s0}`: each 16-bit sample copied to left/right channel slots

- `vTaskDelay(3)` yields CPU each round, ensures `loop()` button polling is not blocked
- After playback ends, send 320 bytes of silence to prevent trailing noise from residual data

5.3.3 Serial Upload Listener

Each `loop()` frame call, continuously monitors serial. First waits for `###UPLOAD###<size>:<filename>\n` command, after receiving, checks remaining space, creates file, replies `READY`; then receives byte-by-byte, writes to SPIFFS in 256-byte batches; after completion, replies `DONE`, after 500ms `ESP.restart()` to rescan files. When SPIFFS is empty, also controls WS2812 to blink white light at 300ms period, prompting user to upload files.

```
void wav_player_check_upload(void) {
    // Empty SPIFFS → white LED slow blink (300ms)
    ...

    while (Serial.available()) {
        char c = Serial.read();

        // --- Command phase: parse ###UPLOAD###size:filename ---
        if (s_up_rem == 0) {
            // Reply READY after receiving command
            ...
        }

        // --- Data phase: buffered write (flush every 256 bytes) ---
        s_up_file.write(s_up_buf, s_up_buf_pos);

        if (s_up_rem == 0) {
            s_up_file.close();
            ESP.restart();
        }
    }
}
```

Protocol format: `###UPLOAD###<filesize>:<filename>\n`. After receiving, check SPIFFS remaining space, if insufficient reply `ERR:SPIFFS full`. Write in 256-byte batches, after completion reply `DONE` and auto-restart.

5.3.4 Public Interface

Function	Description
<code>wav_player_play_file(path)</code>	Start FreeRTOS playback task (8KB stack, priority 5)
<code>wav_player_pause_resume()</code>	Pause/resume toggle
<code>wav_player_stop()</code>	Set <code>s_stop=true</code> to signal task exit
<code>wav_player_is_busy()</code>	Task alive (including paused)
<code>wav_player_is_playing()</code>	Actively streaming (excluding pause)

5.4 rgb_led.cpp — WS2812 RGB LED

WS2812 LED is controlled via Adafruit_NeoPixel library: `led.begin()` initializes, `led.setBrightness(60)` prevents glare, `led.setPixelColor()` + `led.show()` sets color and refreshes. Rainbow effect uses hue ring algorithm, splitting 16-bit hue (0-65535) into 6 sectors, linear interpolation within sectors for smooth color transitions.

```
#include <Adafruit_NeoPixel.h>

static Adafruit_NeoPixel led(1, RGB_LED_PIN, NEO_GRB + NEO_KHZ800); // 1 LED, GPIO48, GRB
byte order, 800kHz
static uint16_t g_hue = 0; // Hue 0-65535

void rgb_led_init(void) {
    led.begin(); // Start NeoPixel library
    led.setBrightness(60); // Brightness 60/255 (anti-glare)
    led.clear(); // All black
    led.show(); // Send data to strip
}

void rgb_led_rainbow_step(void) {
    uint8_t sector = (g_hue >> 13) & 0x7; // Sector 0-5 (top 3 bits)
    uint8_t x = (g_hue >> 5) & 0xFF; // Blend factor 0-255
    uint8_t r, gr, b;

    switch (sector) {
        case 0: r = 255; gr = x; b = 0; break; // Red→Yellow
        case 1: r = 255 - x; gr = 255; b = 0; break; // Yellow→Green
        case 2: r = 0; gr = 255; b = x; break; // Green→Cyan
        case 3: r = 0; gr = 255 - x; b = 255; break; // Cyan→Blue
        case 4: r = x; gr = 0; b = 255; break; // Blue→Magenta
        default: r = 255; gr = 0; b = 255 - x; break; // Magenta→Red
    }
    rgb_led_set(r, gr, b);
    g_hue += 400; // Hue step ~2.2°
}
```

Hue 0-65535 divided into 6 sectors of 60° each (R→Y→G→C→B→M), linear interpolation within sectors for smooth transitions. Each step `g_hue += 400`, about 164 steps to complete a cycle.

5.5 Main Program — Button State Machine and LED Effects

`Arduino_Player_v2.ino` is the entry point of the entire project, `setup()` pulls PA low in sequence, initializes serial, audio chain, SPIFFS, button, and LED. `loop()` implements 1kHz button state machine and rainbow LED control, calling upload listener every frame.

```
#include "app_speech.h"
#include "wav_file_player.h"
#include "rgb_led.h"

#define LONG_PRESS_MS 800 // Long-press threshold 800ms
```

```

void setup() {
    // Pull PA LOW ASAP to suppress reset pop
    pinMode(VOICE_EN_PIN, OUTPUT);
    digitalWrite(VOICE_EN_PIN, 0);

    Serial.begin(115200);           // Serial baud rate 115200
    Serial.setRXBufferSize(8192);  // Enlarge RX buffer (for uploads)
    delay(500);                    // Wait for serial stability

    Codec_I2S_init();              // Audio system init
    wav_player_init();             // SPIFFS mount + file scan
    btn_play_init();               // Button GPIO init
    rgb_led_init(); rgb_led_off(); // LED init + off

    const char *f = wav_player_first_wav();
    if (f) Serial.printf("Play file: %s\n", f);
    else Serial.println("No .wav files in SPIFFS!");

    Serial.println("Short=Play/Stop | Long=End");
    Serial.println("=====\n");
}

```

`loop()` runs at 1kHz (`vTaskDelay(1)`), implements three-layer button logic and rainbow LED: falling edge debounce and record timestamp → rising edge distinguishes short press (<800ms) and long press (≥800ms); if held beyond threshold without release, triggers long press immediately. During playback, rainbow steps every 5 frames (~10ms), auto turns off LED when not playing.

```

void loop() {
    static bool btn_prev = true; // Previous GPIO level
    static bool btn_done = false; // Press already handled
    static uint32_t btn_down_ticks = 0; // Press timestamp (ticks)
    static uint32_t rainbow_tick = 0; // Rainbow step counter

    bool btn_raw = digitalRead(BTN_PLAY_PIN); // HIGH=released, LOW=pressed

    // ===== Falling edge: debounce + record =====
    if (!btn_raw && btn_prev) {
        vTaskDelay(pdMS_TO_TICKS(10)); // 10ms coarse debounce
        if (digitalRead(BTN_PLAY_PIN) == 0) { // Confirm still pressed
            btn_prev = false;
            btn_done = false;
            btn_down_ticks = xTaskGetTickCount(); // Record tick count
        }
    }

    // ===== Rising edge: short/long press =====
    else if (btn_raw && !btn_prev) {
        btn_prev = true;
        if (btn_done) continue; // Long-press handled, skip

        uint32_t held = (xTaskGetTickCount() - btn_down_ticks) * portTICK_PERIOD_MS;
        if (held < LONG_PRESS_MS) { // Short: Play/Pause

```

```

        if (wav_player_is_busy()) wav_player_pause_resume();
        else wav_player_play_file(wav_player_first_wav());
    } else { // Long: Stop
        wav_player_stop(); rgb_led_off();
    }
}
// ===== Immediate long-press (don't wait for release) =====
if (!btn_prev && !btn_done) {
    if ((xTaskGetTickCount() - btn_down_ticks) * portTICK_PERIOD_MS >= LONG_PRESS_MS) {
        btn_done = true; // Mark handled
        wav_player_stop(); rgb_led_off();
    }
}
// ===== Rainbow LED effect =====
if (wav_player_is_playing()) {
    if (++rainbow_tick >= 5) { // Step every 5 loops (~10ms)
        rainbow_tick = 0;
        rgb_led_rainbow_step();
    }
} else {
    rainbow_tick = 0; // Reset when not playing
    rgb_led_off(); // Auto-off when not playing
}

wav_player_check_upload(); // Background upload listener
vTaskDelay(pdMS_TO_TICKS(1)); // 1ms tick
}

```

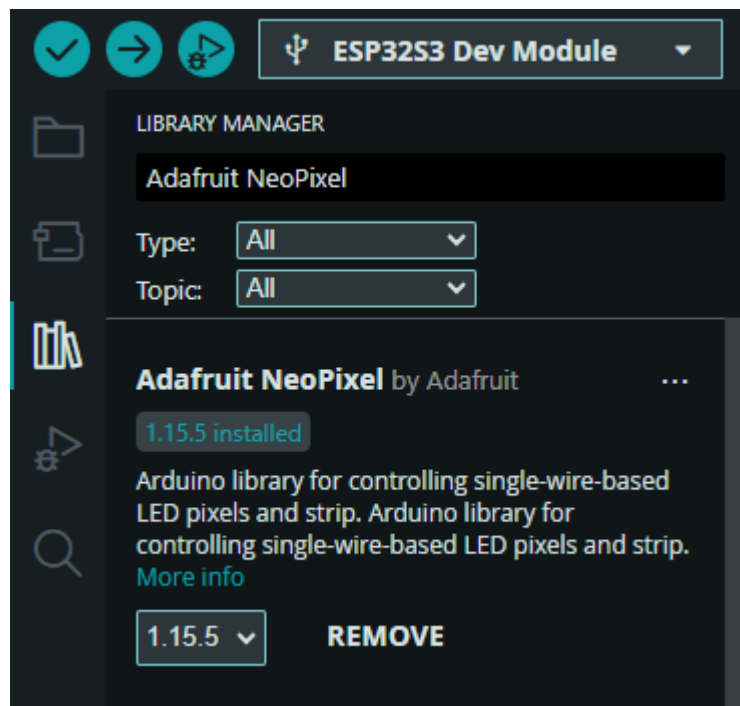
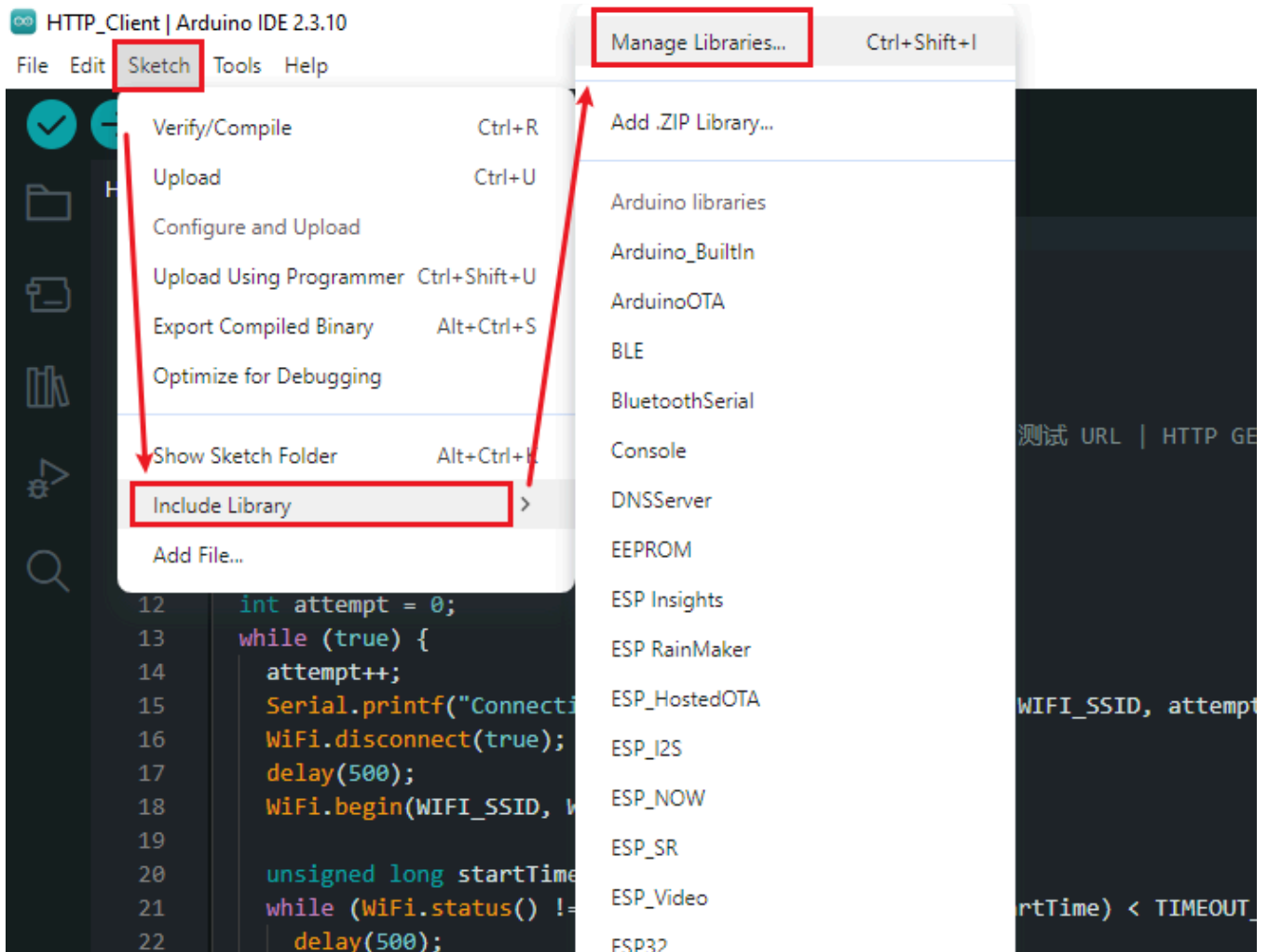
6. Operation Flow

6.1 IDE Configuration and Build

Step 1: Install Dependencies

- Board Manager: install "esp32 by Espressif Systems" (v3.3.8+)
- Library Manager: search "Adafruit NeoPixel", install "Adafruit NeoPixel" (by Adafruit)

Import Library:

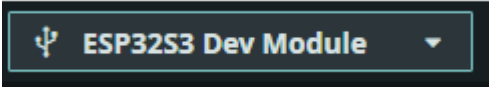


Step 2: Select Configuration

Setting	Value
Board	ESP32S3 Dev Module

Setting	Value
Flash Size	16MB (128Mb)
Partition Scheme	Custom (partitions.csv)
PSRAM	OPI PSRAM

Board Selection



Select Other Board and Port

Select both a Board and a Port if you want to upload a sketch.
If you only select a Board you will be able to compile, but not to upload your sketch.

BOARDS

esp32s3 dev module

ESP32S3 Dev Module ✓

ESP32S3 Dev Module Octal (WROOM2)

PORTS

COM6 Serial Port (USB) ✓

☐ Show all ports

CANCEL

OK

Enable Flash Size: 16MB (128Mb)

- Auto Format Ctrl+T
- Archive Sketch
- Manage Libraries... Ctrl+Shift+I
- Serial Monitor Ctrl+Shift+M
- Serial Plotter
- Firmware Updater
- Upload SSL Root Certificates
- Board: "ESP32S3 Dev Module" >
- Port: "COM4" >
- Reload Board Data
- Get Board Info
- USB CDC On Boot: "Disabled" >
- CPU Frequency: "240MHz (WiFi)" >
- Core Debug Level: "None" >
- USB DFU On Boot: "Disabled" >
- Erase All Flash Before Sketch Upload: "Disabled" >
- Events Run On: "Core 1" >
- Flash Mode: "QIO 80MHz" >
- Flash Size: "16MB (128Mb)"** >
- JTAG Adapter: "Disabled" >
- Arduino Runs On: "Core 1" >
- USB Firmware MSC On Boot: "Disabled" >
- Partition Scheme: "16M Flash (3MB APP/9.9MB FATFS)" >
- PSRAM: "OPI PSRAM" >
- Upload Mode: "UART0 / Hardware CDC" >
- Upload Speed: "921600" >
- USB Mode: "USB-OTG (TinyUSB)" >
- Zigbee Mode: "Disabled" >
- Programmer >
- Burn Bootloader

- 4MB (32Mb)
- 8MB (64Mb)
- ☒ 16MB (128Mb)
- 32MB (256Mb)

Custom Partition:

Arduino_Recorder_USB_MSC | Arduino IDE 2.3.10

File Edit Sketch **Tools** Help

The screenshot shows the Arduino IDE interface with the Tools menu open. The Partition Scheme: "Custom" option is highlighted in the main menu. A red arrow points from this option to the "Custom" option at the bottom of the submenu.

Tools menu items:

- Auto Format (Ctrl+T)
- Archive Sketch
- Manage Libraries... (Ctrl+Shift+I)
- Serial Monitor (Ctrl+Shift+M)
- Serial Plotter
- Firmware Updater
- Upload SSL Root Certificates
- Board: "ESP32S3 Dev Module" >
- Port: "COM4" >
- Reload Board Data
- Get Board Info
- USB CDC On Boot: "Disabled" >
- CPU Frequency: "240MHz (WiFi)" >
- Core Debug Level: "None" >
- USB DFU On Boot: "Disabled" >
- Erase All Flash Before Sketch Upload: "Disabled" >
- Events Run On: "Core 1" >
- Flash Mode: "QIO 80MHz" >
- Flash Size: "16MB (128Mb)" >
- JTAG Adapter: "Disabled" >
- Arduino Runs On: "Core 1" >
- USB Firmware MSC On Boot: "Disabled" >
- Partition Scheme: "Custom" >**
- PSRAM: "OPI PSRAM" >
- Upload Mode: "UART0 / Hardware CDC" >
- Upload Speed: "921600" >
- USB Mode: "USB-OTG (TinyUSB)" >
- Zigbee Mode: "Disabled" >
- Programmer >
- Burn Bootloader

Partition Scheme: "Custom" submenu items:

- Default 4MB with spiiffs (1.2MB APP/1.5MB SPIFFS)
- Default 4MB with ffat (1.2MB APP/1.5MB FATFS)
- 8M with spiiffs (3MB APP/1.5MB SPIFFS)
- Minimal (1.3MB APP/700KB SPIFFS)
- No FS 4MB (2MB APP x2)
- No OTA (2MB APP/2MB SPIFFS)
- No OTA (1MB APP/3MB SPIFFS)
- No OTA (2MB APP/2MB FATFS)
- No OTA (1MB APP/3MB FATFS)
- Huge APP (3MB No OTA/1MB SPIFFS)
- Minimal SPIFFS (1.9MB APP with OTA/128KB SPIFFS)
- 16M Flash (2MB APP/12.5MB FATFS)
- 16M Flash (3MB APP/9.9MB FATFS)
- RainMaker 4MB
- RainMaker 4MB No OTA
- RainMaker 8MB
- 32M Flash (4.8MB APP/22MB FATFS)
- 32M Flash (4.8MB APP/22MB LittleFS)
- 32M Flash (13MB APP/6.75MB SPIFFS)
- ESP SR 16M (3MB APP/6MB SPIFFS/3.9MB MODEL)
- Zigbee ZCZR 4MB with spiiffs
- Zigbee ZCZR 8MB with spiiffs
- Custom**

Enable PSRAM:

The screenshot shows the Arduino IDE interface with the 'Tools' menu open. A red box highlights the 'Tools' menu item in the top bar. Another red box highlights the 'PSRAM: "OPI PSRAM"' option in the Tools menu. A third red box highlights the 'OPI PSRAM' option in the submenu that appears when 'PSRAM: "OPI PSRAM"' is selected. A red arrow points from the 'PSRAM: "OPI PSRAM"' option to its submenu.

The Tools menu contains the following options:

- Auto Format (Ctrl+T)
- Archive Sketch
- Manage Libraries... (Ctrl+Shift+I)
- Serial Monitor (Ctrl+Shift+M)
- Serial Plotter
- Firmware Updater
- Upload SSL Root Certificates
- Board: "ESP32S3 Dev Module" >
- Port: "COM4" >
- Reload Board Data
- Get Board Info
- USB CDC On Boot: "Disabled" >
- CPU Frequency: "240MHz (WiFi)" >
- Core Debug Level: "None" >
- USB DFU On Boot: "Disabled" >
- Erase All Flash Before Sketch Upload: "Disabled" >
- Events Run On: "Core 1" >
- Flash Mode: "QIO 80MHz" >
- Flash Size: "16MB (128Mb)" >
- JTAG Adapter: "Disabled" >
- Arduino Runs On: "Core 1" >
- USB Firmware MSC On Boot: "Disabled" >
- Partition Scheme: "Custom" >
- PSRAM: "OPI PSRAM" >**
 - Disabled
 - QSPI PSRAM
 - ✓ OPI PSRAM**
- Upload Mode: "UART0 / Hardware CDC" >
- Upload Speed: "921600" >
- USB Mode: "USB-OTG (TinyUSB)" >
- Zigbee Mode: "Disabled" >
- Programmer >
- Burn Bootloader

Step 3: Build and Flash

1. Open `Arduino_Player_v2.ino` → Click "Upload"

```
===== Audio Player =====
===== Audio System Init (Playback Only) =====
[PWR] PA pin configured (off) on GPIO7
[I2C] Initialized (SDA:41 SCL:42)
===== I2C Bus Scan =====
      [OK] 0x18
      [OK] 0x3C
===== Found 2 device(s) =====
[I2S] Ready (MCLK:15 BCLK:16 WS:18 DOUT:8)
[ES8311] Hardware init done (slave, ext MCLK)
[ES8311] DAC enabled (FS:16000Hz Vol:75%)
===== Audio Player Ready =====
[SPIFFS] 3228KB total / 0KB used
=== SPIFFS files ===

=====
SPIFFS empty - run flash_spiffs.bat
=====

[LED] init GPIO48
No .wav files in SPIFFS!
Short=Play/Stop | Long=End
=====
```

2. First use requires separate SPIFFS image flash: run `flash_spiffs.bat`

Must close serial monitor!!!

```
=====
ESP32 SPIFFS Flash Tool
=====

[1/2] Creating SPIFFS image...
      Size: 3604480 bytes
      File: Ideal_Thirties.wav (2560044 bytes)

[2/2] Flashing to ESP32-S3...
      (Close Arduino IDE Serial Monitor first!)

Warning: Deprecated: Choice 'default_reset' for option '--before' is deprecated. Use 'default-reset' instead.
esptool v5.2.0
Connected to ESP32-S3 on COM6:
Chip type:          ESP32-S3 (QFN56) (revision v0.2)
Features:           Wi-Fi, BT 5 (LE), Dual Core + LP Core, 240MHz, Embedded PSRAM 8MB (AP_3v3)
Crystal frequency:  40MHz
MAC:                a4:cb:8f:c6:b8:64

Stub flasher running.
Changing baud rate to 921600...
Changed.

Configuring flash size...
Flash will be erased from 0x00090000 to 0x003fffff...
Wrote 3604480 bytes (2570907 compressed) at 0x00090000 in 34.6 seconds (833.2 kbit/s).
Verifying written data...
```

WAV file requirements: 16kHz / 16-bit / mono, place in `data/` directory before flashing.

3. If you need to change serial port, please change it in `flash_spiffs.ps1` file (**besides this, you must also confirm that the file paths of `mkspiffs.exe` and `esptool.exe` in the code all correspond**).

```
文件  编辑  查看   

# ESP32 SPIFFS Flash Tool — mkspiffs + esptool
param(
    [string]$Port = "COM6",
    [string]$Size = "0x370000",
    [string]$Offset = "0x90000"
)

$mkspiffs = "$env:LOCALAPPDATA\Arduino15\packages\esp32\tools\mkspiffs\0.2.3\mkspiffs.exe"
$esptool = "$env:LOCALAPPDATA\Arduino15\packages\esp32\tools\esptool\py\5.2.0\esptool.exe"
$dataDir = Join-Path $PSScriptRoot "data"
$image = Join-Path $env:TEMP "spiffs_v2.bin"

Write-Host "===== " -ForegroundColor Cyan
Write-Host " ESP32 SPIFFS Flash Tool " -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan
Write-Host ""

if (-not (Test-Path $mkspiffs)) {
    Write-Host "[ERROR] mkspiffs.exe not found" -ForegroundColor Red
    Write-Host $mkspiffs
    pause; exit 1
}
if (-not (Test-Path $esptool)) {
    Write-Host "[ERROR] esptool.exe not found" -ForegroundColor Red
    pause; exit 1
}
if (-not (Get-ChildItem "$dataDir\*.wav" 2>$null)) {
    Write-Host "[ERROR] No .wav files in $dataDir" -ForegroundColor Red
    pause; exit 1
}
}
```

6.2 Normal Playback

Power-On Serial Output:

```
===== Audio Player =====
===== Audio System Init (Playback Only) =====
[PWR] PA pin configured (off) on GPIO7
[I2C] Initialized (SDA:41 SCL:42)
===== I2C Bus Scan =====
[OK] 0x18
[OK] 0x3C
===== Found 2 device(s) =====
[I2S] Ready (MCLK:15 BCLK:16 WS:18 DOUT:8)
[ES8311] Hardware init done (slave, ext MCLK)
[ES8311] DAC enabled (FS:16000Hz Vol:75%)
===== Audio Player Ready =====
[SPIFFS] 3228KB total / 2520KB used
=== SPIFFS files ===
    Ideal_Thirties.wav (2560044 bytes)
[LED] init GPIO48
Play file: /Ideal_Thirties.wav
```

Short=Play/Stop | Long=End

=====

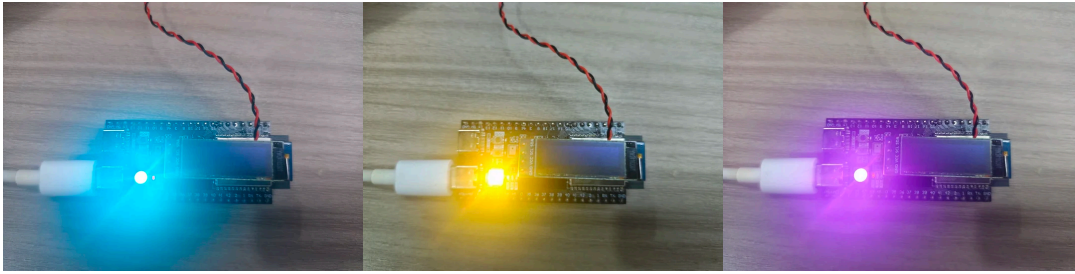
```
===== Audio Player =====
===== Audio System Init (Playback Only) =====
[PWR] PA pin configured (off) on GPIO7
[I2C] Initialized (SDA:41 SCL:42)
===== I2C Bus Scan =====
  [OK] 0x18
  [OK] 0x3C
===== Found 2 device(s) =====
[I2S] Ready (MCLK:15 BCLK:16 WS:18 DOUT:8)
[ES8311] Hardware init done (slave, ext MCLK)
[ES8311] DAC enabled (FS:16000Hz Vol:75%)
===== Audio Player Ready =====
[SPIFFS] 3228KB total / 2520KB used
=== SPIFFS files ===
  Ideal_Thirties.wav (2560044 bytes)
[LED] init GPIO48
Play file: /Ideal_Thirties.wav
Short=Play/Stop | Long=End
=====
```

Expected Behavior:

Action	Expected	LED Status
Short press BOOT (<0.8s)	Start playback	Rainbow flow
Short during play	Pause	Off
Short after pause	Resume	Rainbow flow
Long press BOOT (≥0.8s)	Stop playback	Off
Play to file end	Auto-stop	Off
No files in SPIFFS	White blink prompt	White 300ms blink

Playback:

```
=== WAV File Info ===
  Sample Rate : 16000 Hz
  Channels    : 1 -> stereo
  Data Size   : 2560000 bytes (80.0s)
[WAV] Playing: /Ideal_Thirties.wav
BTN: Play
[WAV] 1.0s / 80.0s
[WAV] 2.0s / 80.0s
[WAV] 3.0s / 80.0s
[WAV] 4.0s / 80.0s
[WAV] 5.0s / 80.0s
[WAV] 6.0s / 80.0s
[WAV] 7.0s / 80.0s
```



Pause:

```
[WAV] Paused  
BTN: Pause
```

Resume:

```
[WAV] Resumed  
BTN: Resume  
[WAV] 20.0s / 80.0s  
[WAV] 21.0s / 80.0s
```

Stop:

```
[WAV] Stop requested  
BTN: Long -> End  
[WAV] Stopped at 31.5s
```

Audio Files Not Downloaded:



Volume Adjustment:

Playback volume is determined by the `VOICE_VOLUME` macro in `app_speech.h`, default value is 75. Higher number means louder speaker, recompile and reflash to hear the difference.


```

1  #define I2C_SCL_PIN 42
2  #define I2C_SDA_PIN 41
3  #define I2S_MCLK 15
4  #define I2S_BCLK 16
5  #define I2S_WS 18
6  #define I2S_DOUT 8
7  #define I2S_DIN -1
8  #define VOICE_EN_PIN 7
9
10 // 音频 | Audio
11 #define SAMPLE_RATE 16000
12 #define MCLK_MULTIPLE 256
13 #define VOICE_VOLUME 75
14 #define BUF_SIZE 1024

```

Why is the upper limit 75, not 100? ES8311 chip has an internal "digital volume knob" called `DACV` register (address 0x32), the `es8311setvolume()` function converts your percentage to 0~255 (e.g. 75% maps to 191 = 0xBF). But this chip has a flaw — above 75%, the sound will "break" and become harsh and distorted. So the code simply caps it: regardless of what you pass, above 75 is calculated as 75. This is not a bug, it's intentional sound quality protection.

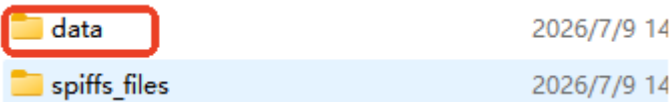
If you really feel the sound is not loud enough, don't hard-change this limit — that's the chip's ceiling, forcing it will just make the sound worse. The correct approach is to adjust hardware: there's a gain resistor on the NS4150 power amplifier chip, changing it can raise the overall volume by a level.

Where to change? Open `app_speech.h`, find:

```
#define VOICE_VOLUME    75           // volume 0-100 (ES8311 max 75)
```

Change 75 to your desired value (any integer between 0 and 75), save, recompile and upload.

6.3 Replace Playback Files

- **Re-flash SPIFFS (recommended):** Put new WAV in `data/` → run `flash_spiffs.bat` → press RST to restart (code always plays first file)
- 
- **Serial upload:** Host sends `###UPLOAD###<size>:<filename>\n` → wait `READY` → send binary data → wait `DONE` auto-restart

6.4 Edge Cases

Scenario	Expected Behavior
Long press BOOT during playback (≥0.8s)	Stop immediately (don't wait for release), send silence frame then close file
Play to file end	Auto-stop, LED off
No WAV files in SPIFFS	White LED slow blink, serial prints warning

Scenario	Expected Behavior
WAV format mismatch	Print error, refuse playback
Incomplete WAV file	Auto-detected and deleted during startup scan
Hold BOOT during power-on	GPIO0 is strapping pin, may enter download mode

7. FAQ

Symptom	Cause	Solution
No sound output	PA not enabled / ES8311 not initialized	Measure GPIO7 should be ~3.3V during playback; confirm serial shows <code>DAC enabled</code>
No I2C device	ES8311 wiring error	Check SCL=GPIO42, SDA=GPIO41
Choppy playback	Task stack insufficient / fragmentation	Increase task stack to 16384; reformat SPIFFS
Wrong playback speed	WAV sample rate mismatch	Confirm file is 16kHz, or modify <code>SAMPLE_RATE</code> macro
Noise/pop	PA enable timing wrong	Confirm <code>setup()</code> pulls PA low first; confirm 7-step anti-pop sequence executed
No files in SPIFFS	Image not flashed	Run <code>flash_spi_ffs.bat</code> (close serial monitor first)
WS2812 not lit	Library not installed / GPIO48 wrong	Install Adafruit_NeoPixel; check wiring
Button no response	GPIO0 strapping conflict	Ensure GPIO0 not pulled low during power-on
<code>ESP_I2S.h</code> not found	Arduino core version too old	Update esp32 core ≥ v3.0.0
SPIFFS flash failed	Tool path incorrect	Modify esptool/mkspiffs paths in <code>flash_spi_ffs.ps1</code>

Appendix A: ES8311 Register Quick Reference (DAC Playback)

Register	Address	Write Value	Function
RESET	0x00	0x80 → 0x3F	Global reset release → clear I2S reset
CLK_MGR1	0x01	0x30 → 0x3F	Master clock → external MCLK
CLK_MGR2	0x02	0x00	pre_div=1, multi=x1

Register	Address	Write Value	Function
CLK_MGR3	0x03	0x10	fs_mode=0, adc_osr=16
CLK_MGR4	0x04	0x20	dac_osr=32
CLK_MGR5	0x05	0x00	adc_div=1, dac_div=1
CLK_MGR6	0x06	bclk_div=4	BCLK = MCLK / 4 = 1.024MHz
SDP_IN	0x09	0x0C	DAC I2S standard mode, 16bit
SDP_OUT	0x0A	0x0C	ADC I2S standard mode, 16bit
SYSTEM3	0x0D	0xFA → 0x01	Analog power config
SYSTEM4	0x0E	0x02	Digital power enable
SYSTEM10	0x14	0x1A	PGA gain + DAC output enable
DAC7	0x37	0x08	DAC ramp start
GPIO1	0x44	0x08 → 0x58	GPIO ref → enable internal DAC ref
DAC_MUTE	0x31	bit6/bit5	Mute control (1=mute, 0=normal)
DAC_VOL	0x32	0xBF (75%)	DAC digital volume

Appendix B: Code Index

File	Lines	Description
Arduino_Player_v2.ino	113	Main: setup + loop + button FSM
app_speech.h	40	Pin macros + API declarations
app_speech.cpp	300+	I2C/I2S init + ES8311 register ops
wav_file_player.h	25	WAV player API declarations
wav_file_player.cpp	250+	SPIFFS mount + playback task + upload
rgb_led.h	15	RGB LED API declarations
rgb_led.cpp	80+	WS2812 driver + rainbow algorithm
partitions.csv	5	Partition table